Title: "Data_698_Research Project - Chun Shing Leung TONY"

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, f1_score, classification_report, roc_curve
from matplotlib.colors import LinearSegmentedColormap
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import roc_auc_score, RocCurveDisplay
import warnings
warnings.filterwarnings('ignore')
```

The Dataset is retreived from Kaggle: https://www.kaggle.com/code/ananthu19/online-payments-fraud-detection/input

```
# Read the CSV file
fraud = pd.read_csv("C:/Users/tonyl/OneDrive/Desktop/CUNY/DATA 698/Data/fraud.csv")
```

```
print(fraud.info())
print(fraud.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column          Dtype
---  ------          -----
 0   step            int64
 1   type            object
 2   amount          float64
 3   nameOrig        object
 4   oldbalanceOrg   float64
 5   newbalanceOrig  float64
 6   nameDest        object
 7   oldbalanceDest  float64
 8   newbalanceDest  float64
 9   isFraud         int64
 10  isFlaggedFraud  int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
None
   step      type    amount     nameOrig  oldbalanceOrg  newbalanceOrig  \
0     1   PAYMENT   9839.64  C1231006815       170136.0       160296.36
1     1   PAYMENT   1864.28  C1666544295        21249.0        19384.72
2     1  TRANSFER    181.00  C1305486145          181.0            0.00
3     1  CASH_OUT    181.00   C840083671          181.0            0.00
4     1   PAYMENT  11668.14  C2048537720        41554.0        29885.86

      nameDest  oldbalanceDest  newbalanceDest  isFraud  isFlaggedFraud
0  M1979787155             0.0             0.0        0               0
1  M2044282225             0.0             0.0        0               0
2   C553264065             0.0             0.0        1               0
3    C38997010         21182.0             0.0        1               0
4  M1230701703             0.0             0.0        0               0
```

```
# Transform Columns to Categorical Variable

fraud['isFraud'] = fraud['isFraud'].astype('category')
fraud['isFlaggedFraud'] = fraud['isFlaggedFraud'].astype('category')
fraud['type'] = fraud['type'].astype('category')

# Check after converting
print(fraud.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column          Dtype
---  ------          -----
 0   step            int64
 1   type            category
 2   amount          float64
 3   nameOrig        object
 4   oldbalanceOrg   float64
 5   newbalanceOrig  float64
 6   nameDest        object
 7   oldbalanceDest  float64
 8   newbalanceDest  float64
 9   isFraud         category
 10  isFlaggedFraud  category
dtypes: category(3), float64(5), int64(1), object(2)
memory usage: 406.5+ MB
None
```

```
# Count Zero Values in Each Column

zero_counts = (fraud == 0).sum().reset_index()
zero_counts.columns = ['Column', 'Zero_Count']
zero_counts = zero_counts.sort_values(by='Zero_Count', ascending=False)

print(zero_counts)
```

```
            Column  Zero_Count
10  isFlaggedFraud     6362604
9          isFraud     6354407
5   newbalanceOrig     3609566
7   oldbalanceDest     2704388
8   newbalanceDest     2439433
4    oldbalanceOrg     2102449
2           amount          16
1             type           0
0             step           0
6         nameDest           0
3         nameOrig           0
```

```
# Remove Transactions Where amount == 0
fraud = fraud[fraud['amount'] != 0]

print(f"Number of rows after removing zero amount transactions: {len(fraud)}")
```

```
Number of rows after removing zero amount transactions: 6362604
```

Exploratory Data Analysis:

```
# Summary statistics
print(fraud.describe(include='all'))
```

```
                step      type        amount     nameOrig  oldbalanceOrg  \
count   6.362604e+06   6362604  6.362604e+06      6362604   6.362604e+06
unique           NaN         5           NaN      6353291            NaN
top              NaN  CASH_OUT           NaN  C1677795071            NaN
freq             NaN   2237484           NaN            3            NaN
mean    2.433965e+02       NaN  1.798624e+05          NaN   8.338852e+05
std     1.423311e+02       NaN  6.038589e+05          NaN   2.888246e+06
min     1.000000e+00       NaN  1.000000e-02          NaN   0.000000e+00
25%     1.560000e+02       NaN  1.338975e+04          NaN   0.000000e+00
50%     2.390000e+02       NaN  7.487288e+04          NaN   1.420858e+04
75%     3.350000e+02       NaN  2.087218e+05          NaN   1.073160e+05
max     7.430000e+02       NaN  9.244552e+07          NaN   5.958504e+07

        newbalanceOrig     nameDest  oldbalanceDest  newbalanceDest  \
count     6.362604e+06      6362604    6.362604e+06    6.362604e+06
unique             NaN      2722359             NaN             NaN
top                NaN  C1286084959             NaN             NaN
freq               NaN          113             NaN             NaN
mean      8.551158e+05          NaN    1.100702e+06    1.224997e+06
std       2.924052e+06          NaN    3.399183e+06    3.674132e+06
min       0.000000e+00          NaN    0.000000e+00    0.000000e+00
25%       0.000000e+00          NaN    0.000000e+00    0.000000e+00
50%       0.000000e+00          NaN    1.327058e+05    2.146616e+05
75%       1.442585e+05          NaN    9.430367e+05    1.111910e+06
max       4.958504e+07          NaN    3.560159e+08    3.561793e+08

          isFraud  isFlaggedFraud
count   6362604.0       6362604.0
unique        2.0             2.0
top           0.0             0.0
freq    6354407.0       6362588.0
mean          NaN             NaN
std           NaN             NaN
min           NaN             NaN
25%           NaN             NaN
50%           NaN             NaN
75%           NaN             NaN
max           NaN             NaN
```

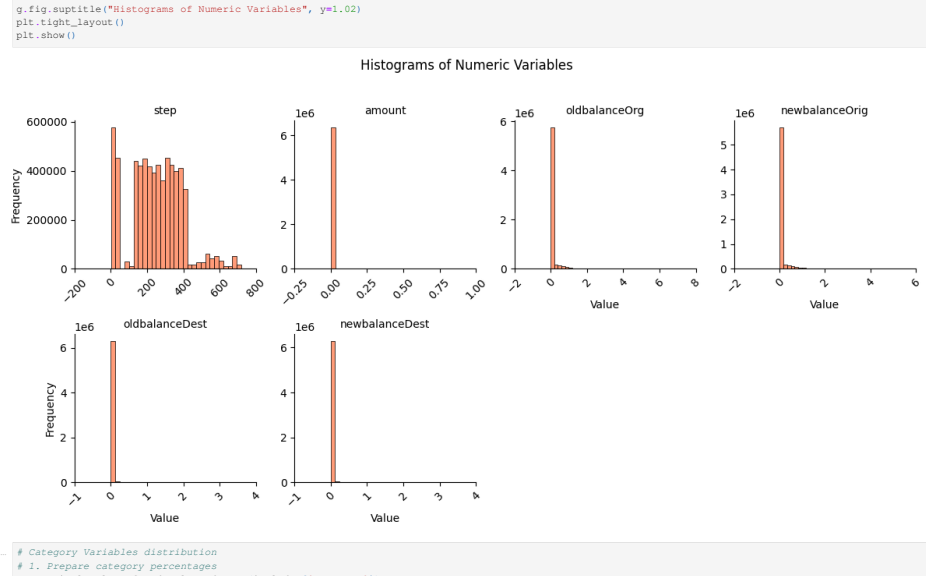
```
# Check for Missing Values
# Total missing values
print(fraud.isna().sum().sum())
```

```
0
```

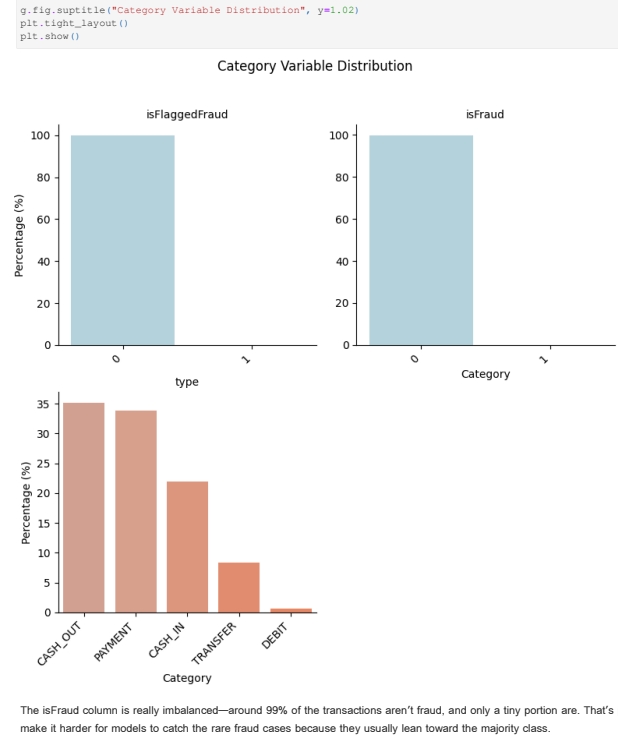
```
# Check for Duplicated Values
# Total duplicated rows
print(fraud.duplicated().sum())
```

```
0
```

```
# Numeric Variable Distribution (Histograms)

# Select numeric columns
numeric_cols = fraud.select_dtypes(include=['number'])

# Reshape data (like pivot_longer in R)
numeric_long = numeric_cols.melt(var_name='Variable', value_name='Value')

# Drop missing
numeric_long = numeric_long.dropna()

# Plot
g = sns.FacetGrid(numeric_long, col='Variable', col_wrap=4, sharex=False, sharey=False)
g.map_dataframe(sns.histplot, x='Value', bins=30, color='coral', edgecolor='black')

g.set_titles("{col_name}")
g.set_axis_labels("Value", "Frequency")
g.set_xticklabels(rotation=45)
g.fig.suptitle("Histograms of Numeric Variables", y=1.02)
plt.tight_layout()
plt.show()
```



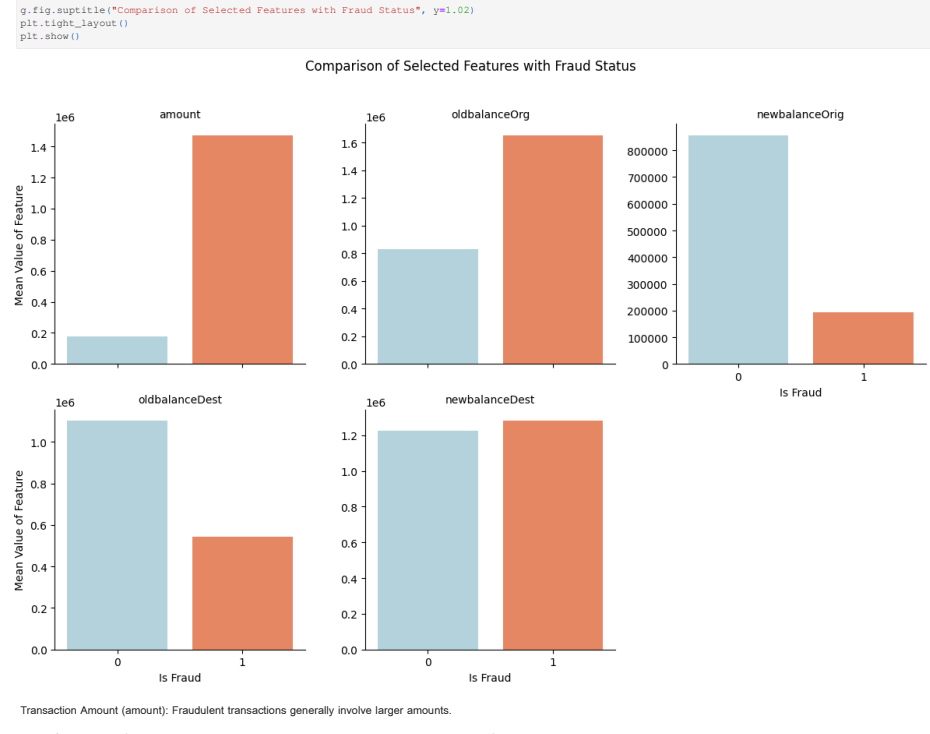
```
# Category Variables distribution
# 1. Prepare category percentages
categorical_cols = fraud.select_dtypes(include=['category'])

category_percentages = (
    categorical_cols.apply(lambda x: x.value_counts(normalize=True) * 100)
    .reset_index()
    .rename(columns={'index': 'Category'})
    .melt(id_vars='Category', var_name='Variable', value_name='Percentage')
    .dropna()
)

# 2. Sort inside each Variable
category_percentages = category_percentages.sort_values(by=['Variable', 'Percentage'], ascending=[True, False])

# 3. Create full color gradient Coral → Light Blue
cmap = LinearSegmentedColormap.from_list("coral_to_lightblue", ["#FF7F50", "#ADD8E6"])
norm = plt.Normalize(category_percentages['Percentage'].min(), category_percentages['Percentage'].max())
full_color_list = [cmap(norm(value)) for value in category_percentages['Percentage']]

# 4. Add color into dataframe
category_percentages['Color'] = full_color_list

# 5. Plot
g = sns.FacetGrid(category_percentages, col="Variable", sharex=False, sharey=False, col_wrap=2, height=4)

# Custom function to plot each facet separately
def barplot_with_gradient(data, **kwargs):
    # Create local color map for this facet
    colors = data['Color'].tolist()
    sns.barplot(
        data=data,
        x="Category", y="Percentage",
        hue="Category",  # <-- Assign hue properly
        dodge=False,
        palette=colors,  # <-- Local colors matching this panel
        legend=False,
        **kwargs
    )

g.map_dataframe(barplot_with_gradient)

g.set_titles("{col_name}")
g.set_axis_labels("Category", "Percentage (%)")

for ax in g.axes.flat:
    for label in ax.get_xticklabels():
        label.set_rotation(45)
        label.set_horizontalalignment('right')

g.fig.suptitle("Category Variable Distribution", y=1.02)
plt.tight_layout()
plt.show()
```



The isFraud column is really imbalanced—around 99% of the transactions aren't fraud, and only a tiny portion are. That's pretty typical for fraud datasets since fraud doesn't happen that often. This kind of imbalance can make it harder for models to catch the rare fraud cases because they usually lean toward the majority class.

Transaction Type Distribution The type variable is skewed, with CASH_OUT and PAYMENT dominating. Although DEBIT and others are less frequent, no changes were made during EDA. Fraud occurs most commonly in CASH_OUT and TRANSFER types.

```
# Select and reshape
fraud_selected = fraud[['isFraud', 'amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']]
fraud_long = fraud_selected.melt(id_vars='isFraud', var_name='Feature', value_name='Value')

# Plot
g = sns.FacetGrid(fraud_long, col="Feature", col_wrap=3, height=4, sharey=False)

g.map_dataframe(
    sns.barplot,
    x="isFraud", y="Value", hue="isFraud",  # <-- Assign hue properly
    estimator='mean',
    errorbar=None,  # <-- Replace ci=None with errorbar=None
    palette=["lightblue", "coral"],
    legend=False
)

g.set_titles("{col_name}")
g.set_axis_labels("Is Fraud", "Mean Value of Feature")

for ax in g.axes.flat:
    for label in ax.get_xticklabels():
        label.set_rotation(0)
        label.set_horizontalalignment('center')

g.fig.suptitle("Comparison of Selected Features with Fraud Status", y=1.02)
plt.tight_layout()
plt.show()
```



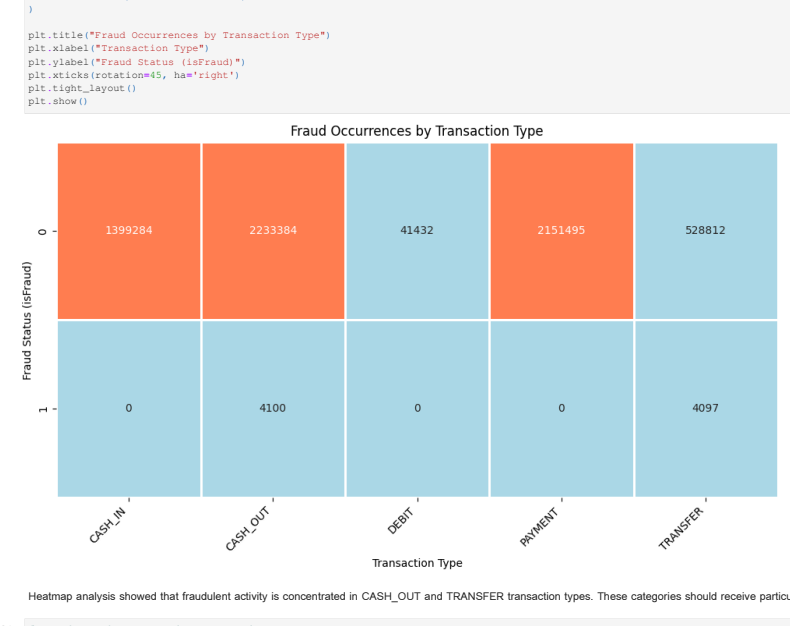
Transaction Amount (amount): Fraudulent transactions generally involve larger amounts.

Sender's Balance Before Transaction (oldbalanceOrg): Higher initial balances are more likely in fraud cases.

Sender's Balance After Transaction (newbalanceOrig): Fraudsters often deplete the sender's balance, leaving little to none.

Receiver's Balance Before Transaction (oldbalanceDest): Fraudulent transactions often target accounts with low or zero initial balances, possibly indicating dummy or less-monitored accounts.

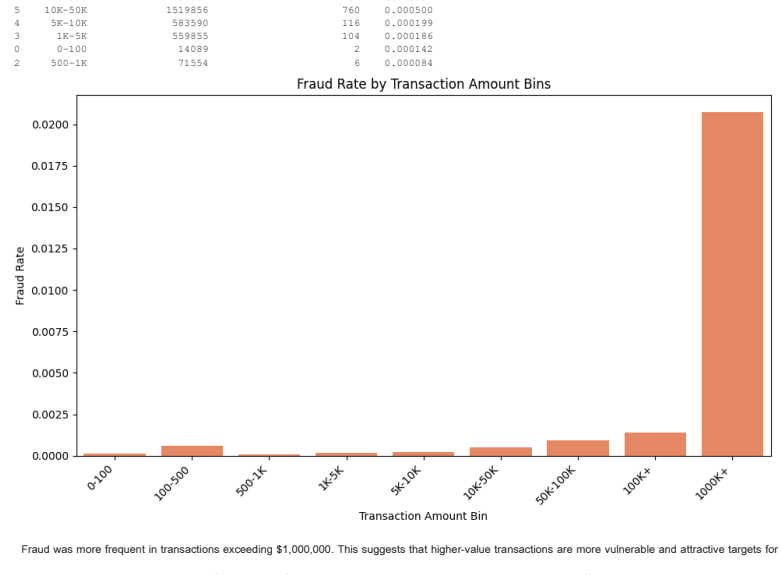
```
# Count fraud occurrences by type
heatmap_data = (
    fraud.groupby(['isFraud', 'type'], observed=True)
    .size()
    .reset_index(name='n')
    .sort_values('n', ascending=False)
)

heatmap_data['isFraud'] = heatmap_data['isFraud'].astype('category')

# Pivot to wide for heatmap
heatmap_pivot = heatmap_data.pivot(index='isFraud', columns='type', values='n').fillna(0)

# Plot
plt.figure(figsize=(10, 6))
sns.heatmap(
    heatmap_pivot,
    annot=True, fmt=".0f", cmap=sns.color_palette(["lightblue","coral"]),
    linewidths=1, linecolor='white', cbar=False
)

plt.title("Fraud Occurrences by Transaction Type")
plt.xlabel("Transaction Type")
plt.ylabel("Fraud Status (isFraud)")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Heatmap analysis showed that fraudulent activity is concentrated in CASH_OUT and TRANSFER transaction types. These categories should receive particular attention in modeling and evaluation.

```
# Fraud Rate by Transaction Amount Bins

# Make sure 'isFraud' is numeric
fraud['isFraud'] = fraud['isFraud'].astype(int)

# Define bins and labels
bins = [0, 100, 500, 1000, 5000, 10000, 50000, 100000, 1000000, float('inf')]
labels = ['0-100', '100-500', '500-1K', '1K-5K', '5K-10K', '10K-50K', '50K-100K', '100K+', '1000K+']

# Bin transaction amounts
fraud['amount_bin'] = pd.cut(fraud['amount'], bins=bins, labels=labels, include_lowest=True)

# Calculate fraud rate per bin
fraud_rate = (
    fraud.groupby('amount_bin', observed=True)
    .agg(
        Total_Transactions=('isFraud', 'count'),
        Fraudulent_Transactions=('isFraud', 'sum')
    )
    .assign(Fraud_Rate=lambda df: df['Fraudulent_Transactions'] / df['Total_Transactions'])
    .reset_index()
    .sort_values('Fraud_Rate', ascending=False)
)

print(fraud_rate)

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=fraud_rate,
    x='amount_bin',
    y='Fraud_Rate',
    color='coral'
)

plt.title("Fraud Rate by Transaction Amount Bins")
plt.xlabel("Transaction Amount Bin")
plt.ylabel("Fraud Rate")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
  amount_bin  Total_Transactions  Fraudulent_Transactions  Fraud_Rate
8     1000K+              130426                     2706    0.020747
7      100K+             2706446                     3800    0.001404
6   50K-100K              714351                      669    0.000937
1    100-500               56987                       34    0.000597
5    10K-50K             1313956                      740    0.000563
4     5K-10K              563590                      116    0.000206
3      1K-5K              559855                      104    0.000186
0      0-100               14089                        2    0.000142
2     500-1K               71554                        4    0.000056
```

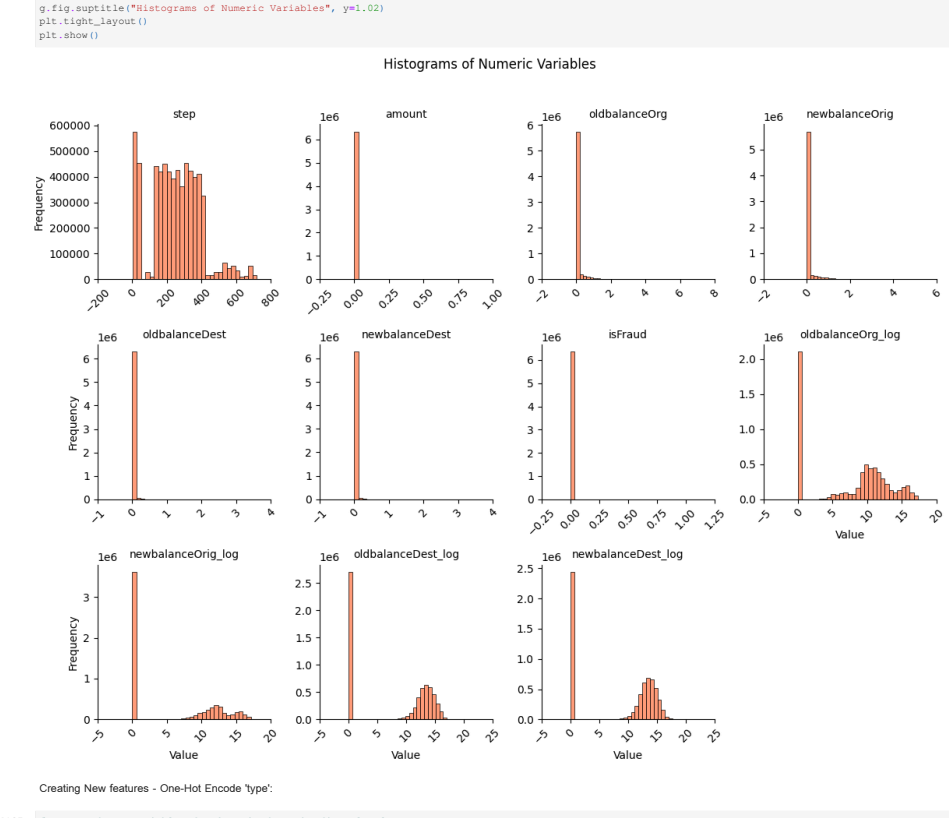


Fraud was more frequent in transactions exceeding $1,000,000. This suggests that higher-value transactions are more vulnerable and attractive targets for fraud.

Feature Engineering: The main reason for these transformations is to handle skewness, outliers, and large-scale differences in numerical data, making it more suitable for EDA and machine learning models.

```
# Apply log1p transformation to prevent log(0) issues
# oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest have highly skewed distributions
# Log transformation reduces skewness, making the distribution more normal-like, which helps models like Logistic Regression
fraud_clean = fraud.copy()

fraud_clean['oldbalanceOrg_log'] = np.log1p(fraud_clean['oldbalanceOrg'])
fraud_clean['newbalanceOrig_log'] = np.log1p(fraud_clean['newbalanceOrig'])
fraud_clean['oldbalanceDest_log'] = np.log1p(fraud_clean['oldbalanceDest'])
fraud_clean['newbalanceDest_log'] = np.log1p(fraud_clean['newbalanceDest'])
```

```
# Numeric Variable Distribution (Histograms)

# Select numeric columns
numeric_cols_clean = fraud_clean.select_dtypes(include=['number'])

# Reshape data
numeric_long_clean = numeric_cols_clean.melt(var_name='Variable', value_name='Value')

# Drop missing
numeric_long_clean = numeric_long_clean.dropna()

# Plot
g = sns.FacetGrid(numeric_long_clean, col='Variable', col_wrap=4, sharex=False, sharey=False)
g.map_dataframe(sns.histplot, x='Value', bins=30, color='coral', edgecolor='black')

g.set_titles("{col_name}")
g.set_axis_labels("Value", "Frequency")
g.set_xticklabels(rotation=45)
g.fig.suptitle("Histograms of Numeric Variables", y=1.02)
plt.tight_layout()
plt.show()
```



Creating New features - One-Hot Encode 'type':

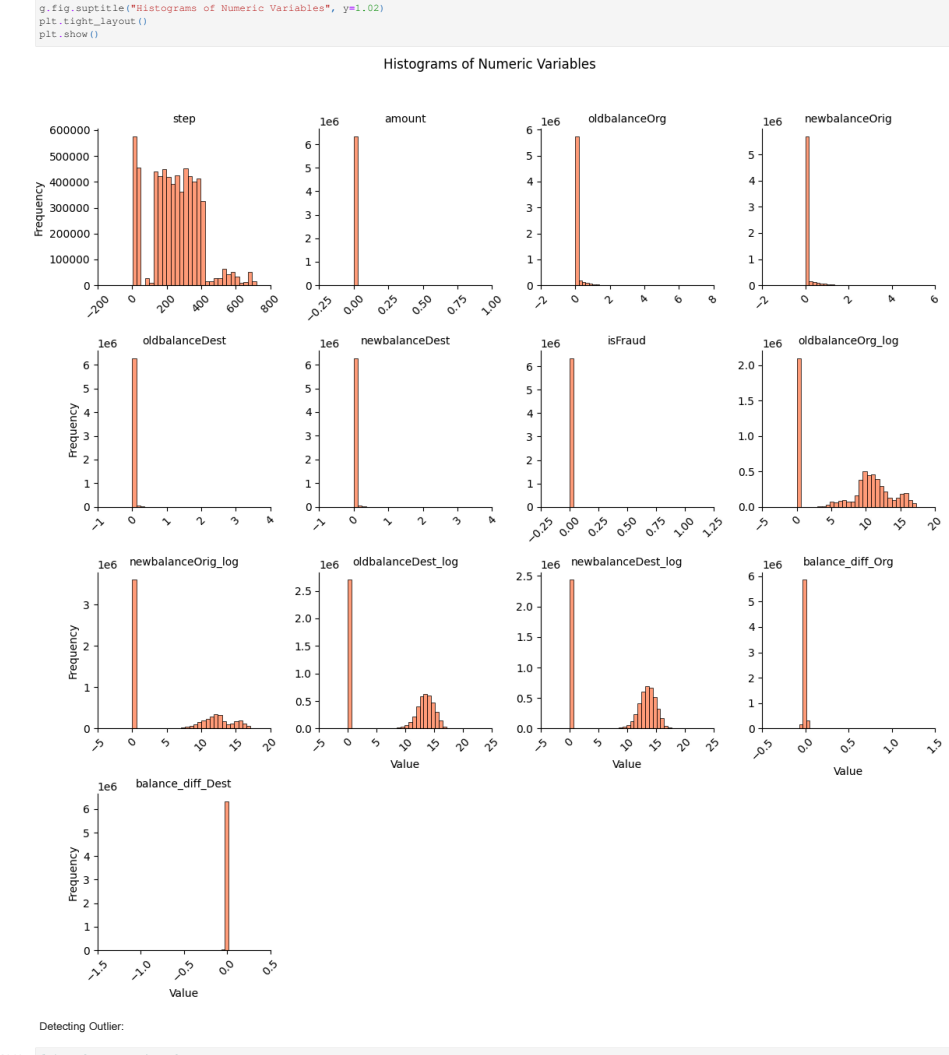
```
# Create dummy variables for 'type', drop the first level
fraud_clean = pd.get_dummies(fraud_clean, columns=['type'], drop_first=True)
```

Two new features are created to see how account balances change during transactions and spot patterns that might point to fraud.

balance_diff_Org shows how much money the sender loses, and balance_diff_Dest shows how much the receiver gains.

```
# Create Balance Difference Features
fraud_clean['balance_diff_Org'] = fraud_clean['oldbalanceOrg'] - fraud_clean['newbalanceOrig']
fraud_clean['balance_diff_Dest'] = fraud_clean['oldbalanceDest'] - fraud_clean['newbalanceDest']
```

```
# Numeric Variable Distribution (Histograms)

# Select numeric columns
numeric_cols_clean2 = fraud_clean.select_dtypes(include=['number'])

# Reshape data (like pivot_longer in R)
numeric_long_clean2 = numeric_cols_clean2.melt(var_name='Variable', value_name='Value')

# Drop missing
numeric_long_clean2 = numeric_long_clean2.dropna()

# Plot
g = sns.FacetGrid(numeric_long_clean2, col='Variable', col_wrap=4, sharex=False, sharey=False)
g.map_dataframe(sns.histplot, x='Value', bins=30, color='coral', edgecolor='black')

g.set_titles("{col_name}")
g.set_axis_labels("Value", "Frequency")
g.set_xticklabels(rotation=45)
g.fig.suptitle("Histograms of Numeric Variables", y=1.02)
plt.tight_layout()
plt.show()
```

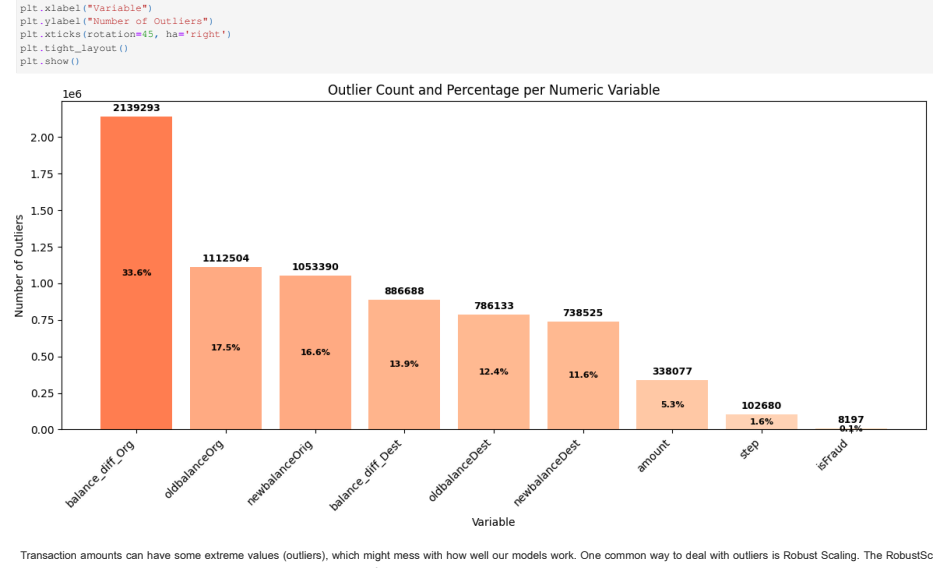


Detecting Outlier:

```
# 1. Select numeric columns
numeric_cols_clean = fraud_clean.select_dtypes(include='number')

# 2. Melt to long format
numeric_long_clean = numeric_cols_clean.melt(var_name='Variable', value_name='Value')

# 3. Drop missing values
numeric_long_clean = numeric_long_clean.dropna()

# 4. Calculate IQR, bounds, and flag outliers
def compute_outliers(df):
    outliers_list = []

    for variable, group in df.groupby('Variable'):
        Q1 = group['Value'].quantile(0.25)
        Q3 = group['Value'].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        group['Q1'] = Q1
        group['Q3'] = Q3
        group['IQR'] = IQR
        group['Lower_Bound'] = lower_bound
        group['Upper_Bound'] = upper_bound
        group['Outlier'] = (group['Value'] < lower_bound) | (group['Value'] > upper_bound)

        # Keep only outliers
        outliers = group[group['Outlier']]
        outliers_list.append(outliers)

    outliers_df = pd.concat(outliers_list)
    return outliers_df

# Apply the function
outliers = compute_outliers(numeric_long_clean)

# 5. Sort by most extreme outlier values
outliers = outliers.sort_values(by='Value', ascending=False)

# Print outliers
print(outliers)
```

```
                   Variable         Value           Q1            Q3  \
37733196     newbalanceDest  3.561793e+08       0.0000  1.111910e+06
31370592     oldbalanceDest  3.560159e+08       0.0000  9.430367e+05
37299406     newbalanceDest  3.560159e+08       0.0000  1.111910e+06
35846600     newbalanceDest  3.555534e+08       0.0000  1.111910e+06
35937267     newbalanceDest  3.555534e+08       0.0000  1.111910e+06
...                     ...           ...          ...           ...
60482648  balance_diff_Dest -8.592310e+07 -149106.0125  0.000000e+00
60037829  balance_diff_Dest -9.244552e+07 -149106.0125  0.000000e+00
60316892  balance_diff_Dest -9.904545e+07 -149106.0125  0.000000e+00
60243774  balance_diff_Dest -1.003252e+08 -149106.0125  0.000000e+00
60488205  balance_diff_Dest -1.056978e+08 -149106.0125  0.000000e+00

                   IQR   Lower_Bound   Upper_Bound  Outlier
37733196  1.111910e+06 -1.667864e+06  2.779774e+06     True
31370592  9.430367e+05 -1.414555e+06  2.357592e+06     True
37299406  1.111910e+06 -1.667864e+06  2.779774e+06     True
35846600  1.111910e+06 -1.667864e+06  2.779774e+06     True
35937267  1.111910e+06 -1.667864e+06  2.779774e+06     True
...                ...           ...           ...      ...
60482648  1.491060e+05 -3.727650e+05  2.236590e+05     True
60037829  1.491060e+05 -3.727650e+05  2.236590e+05     True
60316892  1.491060e+05 -3.727650e+05  2.236590e+05     True
60243774  1.491060e+05 -3.727650e+05  2.236590e+05     True
60488205  1.491060e+05 -3.727650e+05  2.236590e+05     True

[7163487 rows x 8 columns]
```

```
# 1. Summarize outlier count per variable
outlier_summary = (
    outliers.groupby('Variable')
    .agg(Outlier_Count=('Outlier', 'sum'))
    .reset_index()
)

outlier_summary['Total_Count'] = fraud_clean.shape[0]
outlier_summary['Outlier_Percentage'] = (outlier_summary['Outlier_Count'] / outlier_summary['Total_Count']) * 100

# 2. Sort descending
outlier_summary = outlier_summary.sort_values('Outlier_Count', ascending=False)

# 3. Create local Gradient
local_cmap = LinearSegmentedColormap.from_list("local_shades", ["#FFDAB9", "#FF7F50"])  # light coral → deep coral
norm = plt.Normalize(outlier_summary['Outlier_Count'].min(), outlier_summary['Outlier_Count'].max())
bar_colors = [local_cmap(norm(value)) for value in outlier_summary['Outlier_Count']]

# 4. Plot
plt.figure(figsize=(12, 6))
bars = plt.bar(
    outlier_summary['Variable'],
    outlier_summary['Outlier_Count'],
    color=bar_colors
)

# Create the bar plot first
bars = plt.bar(
    outlier_summary['Variable'],
    outlier_summary['Outlier_Count'],
    color=bar_colors
)

# Correct label placement using bar positions
for bar, (_, row) in zip(bars, outlier_summary.iterrows()):
    x = bar.get_x() + bar.get_width() / 2  # Center of the bar
    y_top = bar.get_height()

    # Add Outlier Count label just above the bar
    plt.text(
        x=x,
        y=y_top + (outlier_summary['Outlier_Count'].max() * 0.01),
        s=f"{int(row['Outlier_Count'])}",
        ha='center', va='bottom', fontweight='bold', size=8
    )

    # Add Outlier Percentage label inside the bar
    plt.text(
        x=x,
        y=y_top / 2,
        s=f"{row['Outlier_Percentage']:.1f}%",
        ha='center', va='center', color='black', fontweight='bold', size=8
    )

plt.title("Outlier Count and Percentage per Numeric Variable")
plt.xlabel("Variable")
plt.ylabel("Number of Outliers")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Transaction amounts can have some extreme values (outliers), which might mess with how well our models work. One common way to deal with outliers is Robust Scaling. The RobustScaler helps with this—it provides methods that are more stable when working with outliers or data that isn't normally distributed.

Instead of using the mean and standard deviation, it uses the median and interquartile range (IQR). This makes the data less sensitive to outliers, keeps the relative order of values, and helps models like Logistic Regression, SVM, and Neural Networks perform better—especially since those models can be affected by feature scaling.

```
from sklearn.preprocessing import RobustScaler

# === 1. Set up the RobustScaler ===
scaler = RobustScaler()

# === 2. Fit and transform the selected columns ===
columns_to_scale = ['amount', 'balance_diff_Org', 'balance_diff_Dest']

fraud_clean[['amount_scaled', 'balance_diff_Org_scaled', 'balance_diff_Dest_scaled']] = scaler.fit_transform(
    fraud_clean[columns_to_scale]
)

# === 3. Check summary stats after scaling ===
print(fraud_clean[['amount_scaled', 'balance_diff_Org_scaled', 'balance_diff_Dest_scaled']].describe())
```

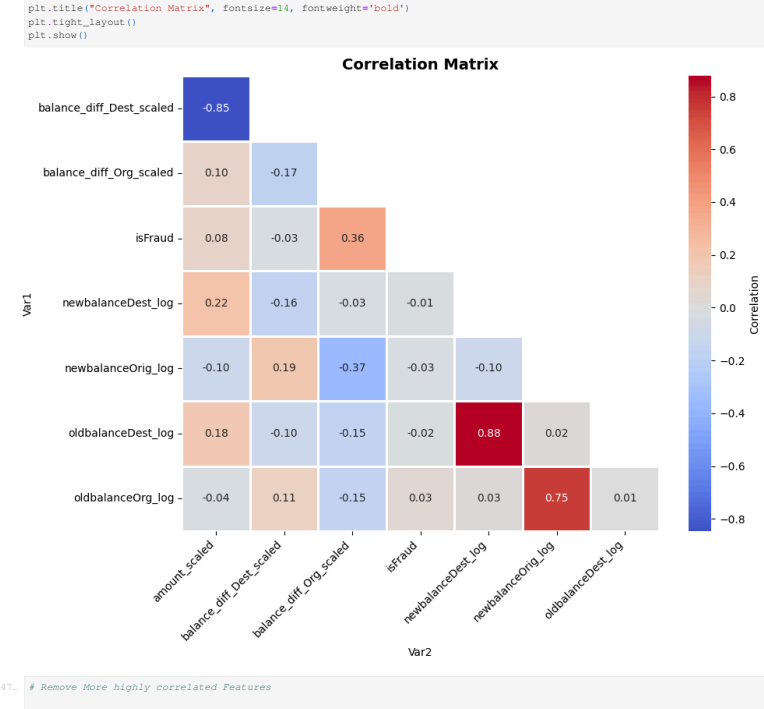
```
       amount_scaled  balance_diff_Org_scaled  balance_diff_Dest_scaled
count   6.362604e+06             6.362604e+06              6.362604e+06
mean    5.374923e-01            -2.291377e+00             -8.108810e-01
std     3.091464e+00             1.446697e+01              5.452295e+00
min    -3.833107e-01            -1.886864e+02             -7.088100e+02
25%    -3.147621e-01             0.000000e+00             -1.000000e+00
50%     0.000000e+00             0.000000e+00              0.000000e+00
75%     6.852273e-01             1.000000e+00              0.000000e+00
max     4.728903e+02             8.851700e+02              8.759423e+01
```

```
fraud_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 6362604 entries, 0 to 6362619
Data columns (total 24 columns):
 #   Column                    Dtype
---  ------                    -----
 0   step                      int64
 1   amount                    float64
 2   nameOrig                  object
 3   oldbalanceOrg             float64
 4   newbalanceOrig            float64
 5   nameDest                  object
 6   oldbalanceDest            float64
 7   newbalanceDest            float64
 8   isFraud                   int64
 9   isFlaggedFraud            category
 10  amount_bin                category
 11  oldbalanceOrg_log         float64
 12  newbalanceOrig_log        float64
 13  oldbalanceDest_log        float64
 14  newbalanceDest_log        float64
 15  type_CASH_OUT             bool
 16  type_DEBIT                bool
 17  type_PAYMENT              bool
 18  type_TRANSFER             bool
 19  balance_diff_Org          float64
 20  balance_diff_Dest         float64
 21  amount_scaled             float64
 22  balance_diff_Org_scaled   float64
 23  balance_diff_Dest_scaled  float64
dtypes: bool(4), category(2), float64(14), int64(2), object(2)
memory usage: 958.7+ MB
```

```
# Drop specified variables
columns_to_drop = [
    "step", "amount", "oldbalanceOrg", "newbalanceOrig",
    "newbalanceDest", "oldbalanceDest", "isFlaggedFraud",
    "nameOrig", "nameDest", "balance_diff_Org", "balance_diff_Dest", "amount_bin"
]

fraud_clean_final = fraud_clean.drop(columns=columns_to_drop)
```

```
# 1. Compute Correlation Matrix (for numeric columns only)
cor_matrix = fraud_clean_final.select_dtypes(include='number').corr()

# 2. Melt into long format
cor_long = cor_matrix.reset_index().melt(id_vars='index')
cor_long.columns = ['Var1', 'Var2', 'value']

# 3. Only keep lower triangle
cor_long = cor_long[cor_long['Var1'] > cor_long['Var2']]

# 4. Convert pivot
pivot_table = cor_long.pivot(index='Var1', columns='Var2', values='value')

# 5. Plot
plt.figure(figsize=(10, 8))
sns.heatmap(
    pivot_table,
    annot=True, fmt=".2f",
    cmap="coolwarm",
    center=0,
    linewidths=1,
    linecolor='white',
    cbar_kws={"label": "Correlation"}
)

plt.xticks(rotation=45, ha='right')
plt.title("Correlation Matrix", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
# Remove More highly correlated Features

columns_to_drop_2 = ["oldbalanceOrg_log", "oldbalanceDest_log", "balance_diff_Dest_scaled"]

fraud_clean_final_2 = fraud_clean_final.drop(columns=columns_to_drop_2)
```

```
fraud_clean_final_2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 6362604 entries, 0 to 6362619
Data columns (total 9 columns):
 #   Column                   Dtype
---  ------                   -----
 0   isFraud                  int64
 1   newbalanceOrig_log       float64
 2   newbalanceDest_log       float64
 3   type_CASH_OUT            bool
 4   type_DEBIT               bool
 5   type_PAYMENT             bool
 6   type_TRANSFER            bool
 7   amount_scaled            float64
 8   balance_diff_Org_scaled  float64
dtypes: bool(4), float64(4), int64(1)
memory usage: 315.5 MB
```

```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# === 1. Prepare X (drop target 'isFraud')
X = fraud_clean_final_2.drop(columns=['isFraud'])

# === 2. Convert bool columns to int (True → 1, False → 0)
bool_cols = X.select_dtypes(include='bool').columns
X[bool_cols] = X[bool_cols].astype(int)

# === 3. Force all to numeric (safety)
X = X.apply(pd.to_numeric, errors='coerce')

# === 4. Add constant intercept
X_const = sm.add_constant(X)

# === 5. Calculate VIF
vif_data = pd.DataFrame()
vif_data["Variable"] = X_const.columns
vif_data["VIF"] = [variance_inflation_factor(X_const.values, i) for i in range(X_const.shape[1])]

# === 6. Drop constant if you want
vif_data = vif_data[vif_data["Variable"] != "const"]

# === 7. Display
print(vif_data)
```

```
                  Variable       VIF
1       newbalanceOrig_log  2.549694
2       newbalanceDest_log  5.692445
3            type_CASH_OUT  5.098792
4               type_DEBIT  1.091785
5             type_PAYMENT  6.390565
6            type_TRANSFER  2.480504
7            amount_scaled  1.191040
8  balance_diff_Org_scaled  1.436996
```

```
# === 1. Prepare X (drop target 'isFraud')
X = fraud_clean_final_2.drop(columns=['isFraud'])

# === 2. Convert bool columns to int (True → 1, False → 0)
bool_cols = X.select_dtypes(include='bool').columns
X[bool_cols] = X[bool_cols].astype(int)

# === 3. Drop 'type_PAYMENT' (highest VIF variable)
X = X.drop(columns=['type_PAYMENT'])

# === 4. Force all to numeric (safety step)
X = X.apply(pd.to_numeric, errors='coerce')

# === 5. Add constant intercept (recommended for VIF calc)
X_const = sm.add_constant(X)

# === 6. Recalculate VIF
vif_data = pd.DataFrame()
vif_data["Variable"] = X_const.columns
vif_data["VIF"] = [variance_inflation_factor(X_const.values, i) for i in range(X_const.shape[1])]

# === 7. Drop constant ('const') from final VIF table
vif_data = vif_data[vif_data["Variable"] != "const"]

# === 8. Display final VIFs
print(vif_data)
```

```
                  Variable       VIF
1       newbalanceOrig_log  2.191345
2       newbalanceDest_log  3.000904
3            type_CASH_OUT  4.241151
4               type_DEBIT  1.048871
5            type_TRANSFER  2.433667
6            amount_scaled  1.187063
7  balance_diff_Org_scaled  1.305951
```

VIF < 5: Typically considered acceptable. No serious multicollinearity concern.

VIF between 5 and 10: Moderate multicollinearity. Use caution—consider dropping or combining variables.

VIF > 10: Strong multicollinearity. You should consider dropping or transforming the variable.

type_PAYMENT will be dropped since it has 6.390565 VIF scores

```
# 1. Drop 'type_PAYMENT' column
fraud_clean_final_2 = fraud_clean_final.drop(columns=["type_PAYMENT"])

# 2. Check structure
fraud_clean_final_2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 6362604 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column                    Dtype
---  ------                    -----
 0   isFraud                   int64
 1   oldbalanceOrg_log         float64
 2   newbalanceOrig_log        float64
 3   oldbalanceDest_log        float64
 4   newbalanceDest_log        float64
 5   type_CASH_OUT             bool
 6   type_DEBIT                bool
 7   type_TRANSFER             bool
 8   amount_scaled             float64
 9   balance_diff_Org_scaled   float64
 10  balance_diff_Dest_scaled  float64
dtypes: bool(3), float64(7), int64(1)
memory usage: 455.1 MB
```

Data Split and Preparation:

```
# Split into training and testing sets (stratified)
X = fraud_clean_final_2.drop(columns=["isFraud"])
y = fraud_clean_final_2["isFraud"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=123,
    stratify=y
)
```

```
# Apply SMOTE to Handle Imbalanced Classes
from imblearn.over_sampling import SMOTE

# === Apply SMOTE (oversampling minority class)
smote = SMOTE(k_neighbors=5, sampling_strategy='auto', random_state=123)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

```
# === 1. Combine X and y
train_data_smote_balanced = pd.DataFrame(X_train_smote)
train_data_smote_balanced['isFraud'] = y_train_smote

# === 2. Map 0/1 to "no"/"yes"
train_data_smote_balanced['isFraud'] = train_data_smote_balanced['isFraud'].map({0: 'no', 1: 'yes'})
y_test = y_test.map({0: 'no', 1: 'yes'})
```

```
# Check structure
print(train_data_smote_balanced['isFraud'].value_counts())
print(y_test.value_counts())
```

```
isFraud
no     5083525
yes    5083525
Name: count, dtype: int64
isFraud
no     1270882
yes       1639
Name: count, dtype: int64
```

Building Models:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold
from sklearn.feature_selection import RFECV
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

# === 1. Prepare training X and y
X_train_final = train_data_smote_balanced.drop(columns=['isFraud'])
y_train_final = train_data_smote_balanced['isFraud']

# Encode labels
y_train_encoded = y_train_final.map({'no': 0, 'yes': 1})
y_test_encoded = y_test.map({'no': 0, 'yes': 1})

# === 2. RFECV (feature selection with class_weight="balanced")
rfecv = RFECV(
    estimator=LogisticRegression(max_iter=1000, random_state=123, class_weight='balanced'),  # add balanced
    step=1,
    cv=StratifiedKFold(5),
    scoring='roc_auc',
    n_jobs=-1
)

# Fit feature selector
rfecv.fit(X_train_final, y_train_encoded)

# Selected features
selected_features = X_train_final.columns[rfecv.support_]
print("\nSelected features after RFECV:", list(selected_features))

# === 3. Refit logistic regression using selected features
X_train_selected = X_train_final[selected_features]
X_test_selected = X_test[selected_features]

# Final Logistic Regression model (with class_weight="balanced")
log_model_selected = LogisticRegression(max_iter=1000, random_state=123, class_weight="balanced")
log_model_selected.fit(X_train_selected, y_train_encoded)
```

```
Selected features after RFECV: ['oldbalanceOrg_log', 'newbalanceOrig_log', 'oldbalanceDest_log', 'newbalanceDest_log', 'type_CASH_OUT', 'type_DEBIT', 'type_TRANSFER', 'amount_scaled', 'balance_diff_Org_scaled', 'balance_diff_Dest_scaled']
```

```
LogisticRegression
LogisticRegression(class_weight='balanced', max_iter=1000, random_state=123)
```

```
# === 4. Predict on test set
y_pred_prob = log_model_selected.predict_proba(X_test_selected)[:, 1]

# Default threshold 0.5
y_pred_class = (y_pred_prob > 0.5).astype(int)

# === 5. Evaluate
print("\n Confusion Matrix:")
print(confusion_matrix(y_test_encoded, y_pred_class))

print("\n Classification Report:")
print(classification_report(y_test_encoded, y_pred_class, target_names=["no", "yes"]))

auc_score = roc_auc_score(y_test_encoded, y_pred_prob)
print(f"\n AUC Score: {auc_score:.4f}")
```

```
 Confusion Matrix:
[[1230095   40787]
 [     33    1606]]

 Classification Report:
              precision    recall  f1-score   support

          no       1.00      0.97      0.98   1270882
         yes       0.04      0.98      0.07      1639

    accuracy                           0.97   1272521
   macro avg       0.52      0.97      0.53   1272521
weighted avg       1.00      0.97      0.98   1272521


 AUC Score: 0.9947
```

```
from sklearn.naive_bayes import GaussianNB

# === Separate X and y for training
X_train_final = train_data_smote_balanced.drop(columns=["isFraud"])
y_train_final = train_data_smote_balanced["isFraud"].map({"no": 0, "yes": 1})  # encode

# Separate X_test
X_test_final = X_test  # (already prepared earlier)

# === Train Naive Bayes
nb_model = GaussianNB()
nb_model.fit(X_train_final, y_train_final)
```

```
GaussianNB
GaussianNB()
```

```
# === Predict class
nb_pred_class = nb_model.predict(X_test_final)

# === Predict probability for the positive class (fraud = 'yes')
nb_pred_prob = nb_model.predict_proba(X_test_final)[:, 1]
```

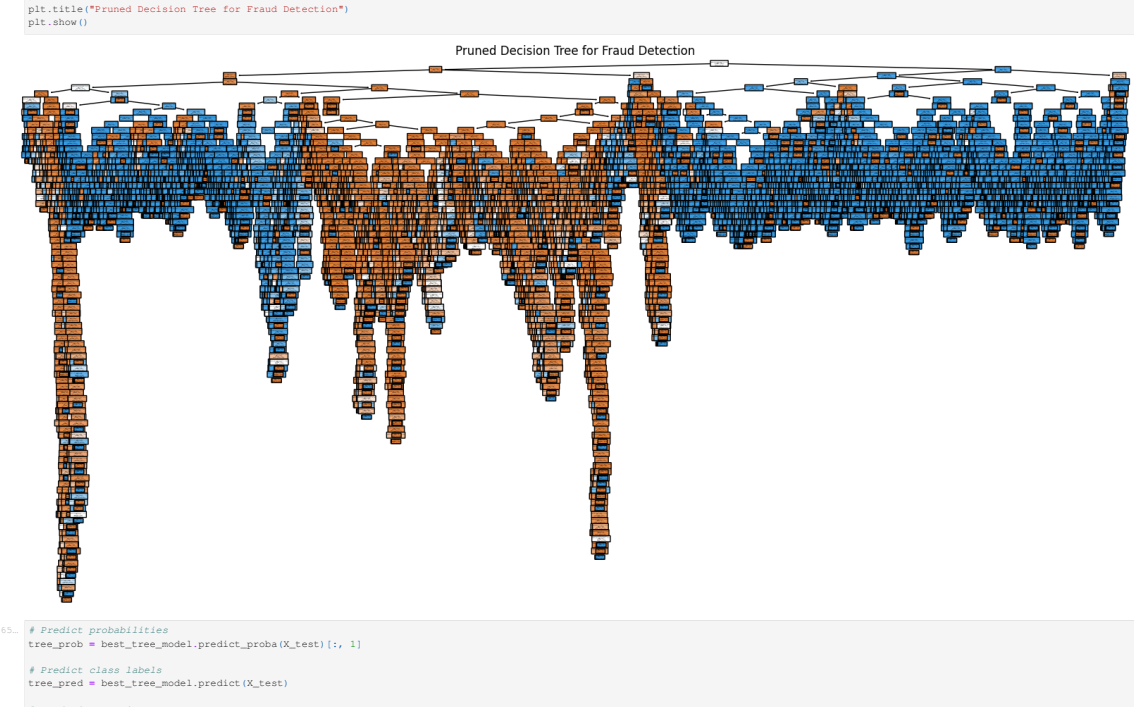
```
# === Confusion Matrix
cm_nb = confusion_matrix(y_test_encoded, nb_pred_class)
print("Confusion Matrix:\n", cm_nb)

# === Full classification report
print(classification_report(y_test_encoded, nb_pred_class, target_names=["no", "yes"]))

# === AUC
auc_nb = roc_auc_score(y_test_encoded, nb_pred_prob)
print("AUC Score:", auc_nb)
```

```
Confusion Matrix:
 [[1220378   50504]
 [    282    1357]]
              precision    recall  f1-score   support

          no       1.00      0.96      0.98   1270882
         yes       0.03      0.83      0.05      1639

    accuracy                           0.96   1272521
   macro avg       0.51      0.89      0.52   1272521
weighted avg       1.00      0.96      0.98   1272521

AUC Score: 0.9767555475897573
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# === Separate X and y
X_train_final = train_data_smote_balanced.drop(columns=["isFraud"])
y_train_final = train_data_smote_balanced["isFraud"].map({"no": 0, "yes": 1})  # Encode

# === Set up Decision Tree
tree_model = DecisionTreeClassifier(random_state=123)

# === Tune tree using cross-validation
param_grid = {"ccp_alpha": [0.0, 0.001, 0.005, 0.01, 0.02, 0.05]}  # cp in R → ccp_alpha in Python
tree_cv = GridSearchCV(
    estimator=tree_model,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)

tree_cv.fit(X_train_final, y_train_final)

# Best model (pruned)
best_tree_model = tree_cv.best_estimator_

print("Best ccp_alpha (cp):", tree_cv.best_params_)
```

```
Best ccp_alpha (cp): {'ccp_alpha': 0.0}
```

```
from sklearn import tree

# Plot decision tree
plt.figure(figsize=(20, 10))
tree.plot_tree(
    best_tree_model,
    filled=True,
    feature_names=X_train_final.columns,
    class_names=["no", "yes"],
    rounded=True,
    proportion=True
)
plt.title("Pruned Decision Tree for Fraud Detection")
plt.show()
```



```
# Predict probabilities
tree_prob = best_tree_model.predict_proba(X_test)[:, 1]

# Predict class labels
tree_pred = best_tree_model.predict(X_test)

# Confusion Matrix
cm_tree = confusion_matrix(y_test_encoded, tree_pred)
print("Confusion Matrix:\n", cm_tree)

# Full classification report
print(classification_report(y_test_encoded, tree_pred, target_names=["no", "yes"]))

# AUC
auc_tree = roc_auc_score(y_test_encoded, tree_prob)
print("AUC Score:", auc_tree)
```

```
Confusion Matrix:
 [[1270241     641]
 [     45    1594]]
              precision    recall  f1-score   support

          no       1.00      1.00      1.00   1270882
         yes       0.71      0.97      0.82      1639

    accuracy                           1.00   1272521
   macro avg       0.86      0.99      0.91   1272521
weighted avg       1.00      1.00      1.00   1272521

AUC Score: 0.9860199330808131
```

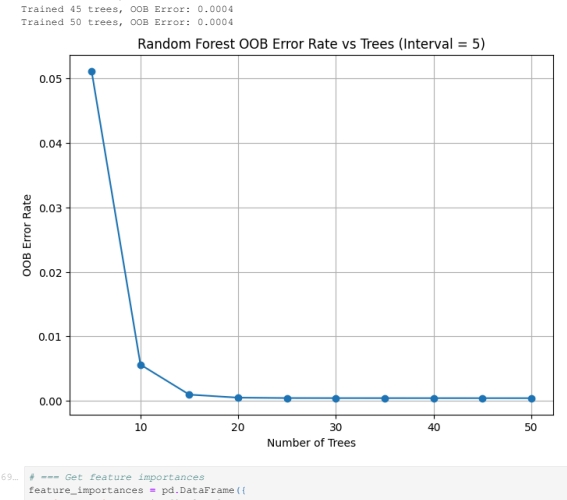
```
from sklearn.ensemble import RandomForestClassifier

# === Separate X and y
X_train_final = train_data_smote_balanced.drop(columns=["isFraud"])
y_train_final = train_data_smote_balanced["isFraud"].map({"no": 0, "yes": 1})  # Encode

# === Fit Random Forest
rf_model = RandomForestClassifier(
    n_estimators=50,          # Number of trees
    oob_score=True,
    random_state=12345,
    n_jobs=-1,
    verbose=1
)

rf_model.fit(X_train_final, y_train_final)
```

```
[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 24 concurrent workers.
[Parallel(n_jobs=-1)]: Done   2 tasks      | elapsed:   53.9s
[Parallel(n_jobs=-1)]: Done  50 out of  50 | elapsed:  2.3min finished
```

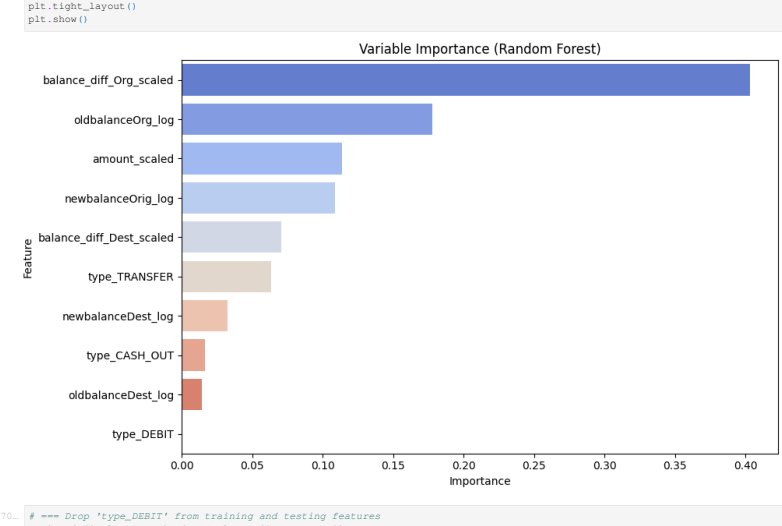
```
RandomForestClassifier
RandomForestClassifier(n_estimators=50, n_jobs=-1, oob_score=True,
                       random_state=12345, verbose=1)
```

```
# === Initialize empty list to store OOB scores
oob_errors = []

# === Initialize model
rf_warm = RandomForestClassifier(
    n_estimators=5,  # Start with 5 tree
    oob_score=True,
    warm_start=True,
    random_state=1234,
    n_jobs=-1
)

# === Prepare X and y
X_train_final = train_data_smote_balanced.drop(columns=["isFraud"])
y_train_final = train_data_smote_balanced["isFraud"].map({"no": 0, "yes": 1})

# === Manually grow trees
n_trees_max = 50
step_size = 5

for i in range(step_size, n_trees_max + 1, step_size):
    rf_warm.set_params(n_estimators=i)  # Increase number of trees
    rf_warm.fit(X_train_final, y_train_final)

    # Save current OOB error
    oob_error = 1 - rf_warm.oob_score_
    oob_errors.append(oob_error)

    print(f"Trained {i} trees, OOB Error: {oob_error:.4f}")

# === Plot OOB Error vs Number of Trees
plt.figure(figsize=(8,6))
plt.plot(range(step_size, n_trees_max + 1, step_size), oob_errors, marker='o')
plt.xlabel("Number of Trees")
plt.ylabel("OOB Error Rate")
plt.title("Random Forest OOB Error Rate vs Trees (Interval = 5)")
plt.grid(True)
plt.show()
```

```
Trained 5 trees, OOB Error: 0.0512
Trained 10 trees, OOB Error: 0.0056
Trained 15 trees, OOB Error: 0.0010
Trained 20 trees, OOB Error: 0.0005
Trained 25 trees, OOB Error: 0.0004
Trained 30 trees, OOB Error: 0.0004
Trained 35 trees, OOB Error: 0.0004
Trained 40 trees, OOB Error: 0.0004
Trained 45 trees, OOB Error: 0.0004
Trained 50 trees, OOB Error: 0.0004
```



```
# === Get feature importances
feature_importances = pd.DataFrame({
    "Feature": X_train_final.columns,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

# === Plot
plt.figure(figsize=(10,6))
sns.barplot(x="Importance", y="Feature", data=feature_importances, palette="coolwarm")
plt.title("Variable Importance (Random Forest)")
plt.tight_layout()
plt.show()
```



```
# === Drop "type_DEBIT" from training and testing features
train_rf_final = X_train.drop(columns=["type_DEBIT"])
test_rf_final = X_test.drop(columns=["type_DEBIT"])

# === Map labels correctly
y_train_rf_final = y_train.replace({'no': 0, 'yes': 1}).astype(int)
y_test_rf_final = y_test.replace({'no': 0, 'yes': 1}).astype(int)

# === Fit Random Forest model
rf_final_model = RandomForestClassifier(
    n_estimators=20,
    oob_score=True,
    random_state=1234,
    n_jobs=-1,
    verbose=1
)

rf_final_model.fit(train_rf_final, y_train_rf_final)
```

```
[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 24 concurrent workers.
[Parallel(n_jobs=-1)]: Done  15 out of  20 | elapsed:   23.4s remaining:    7.8s
[Parallel(n_jobs=-1)]: Done  20 out of  20 | elapsed:   27.5s finished
```

```
RandomForestClassifier
RandomForestClassifier(n_estimators=20, n_jobs=-1, oob_score=True,
                       random_state=1234, verbose=1)
```

```
# === Predict class and probabilities
rf_pred_class = rf_final_model.predict(test_rf_final)
rf_pred_prob = rf_final_model.predict_proba(test_rf_final)[:, 1]

# === Confusion Matrix
cm_rf = confusion_matrix(y_test_rf_final, rf_pred_class)
print("Confusion Matrix:\n", cm_rf)

# === Classification Report
print(classification_report(y_test_rf_final, rf_pred_class, target_names=["no", "yes"]))

# === AUC
auc_rf = roc_auc_score(y_test_rf_final, rf_pred_prob)
print("AUC Score:", auc_rf)
```

```
[Parallel(n_jobs=20)]: Using backend ThreadingBackend with 20 concurrent workers.
[Parallel(n_jobs=20)]: Done   2 out of  20 | elapsed:    0.5s remaining:    0.7s
[Parallel(n_jobs=20)]: Done  20 out of  20 | elapsed:    0.5s finished
[Parallel(n_jobs=20)]: Using backend ThreadingBackend with 20 concurrent workers.
[Parallel(n_jobs=20)]: Done   2 out of  20 | elapsed:    0.4s remaining:    0.6s
[Parallel(n_jobs=20)]: Done  20 out of  20 | elapsed:    0.5s finished
Confusion Matrix:
 [[1270828      54]
 [    313    1326]]
              precision    recall  f1-score   support

          no       1.00      1.00      1.00   1270882
         yes       0.96      0.81      0.88      1639

    accuracy                           1.00   1272521
   macro avg       0.98      0.90      0.94   1272521
weighted avg       1.00      1.00      1.00   1272521

AUC Score: 0.9865124382558469
```

```
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV

# === 1. Prepare Train Data
X_train_svm = train_data_smote_balanced.drop(columns=["isFraud"])
y_train_svm = train_data_smote_balanced["isFraud"].map({"no": 0, "yes": 1}).astype(int)

# === 2. Prepare Test Data
X_test_svm = X_test.copy()
y_test_svm = y_test.replace({'no': 0, 'yes': 1}).astype(int)

# === 3. Align columns
X_test_svm = X_test_svm[X_train_svm.columns]

# === 4. Standardize
scaler = StandardScaler()
X_train_svm_scaled = scaler.fit_transform(X_train_svm)
X_test_svm_scaled = scaler.transform(X_test_svm)

# === 5. Define SGDClassifier with class_weight="balanced" 🎯
sgd = SGDClassifier(
    loss="hinge",         # Linear SVM
    penalty="l2",
    class_weight="balanced",  # 🎯 to adjust for imbalance
    max_iter=1000,
    random_state=12378,
    tol=1e-3
)

# === 6. Hyperparameter search: tune alpha
param_dist = {
    "alpha": np.logspace(-5, 0, 20)  # Try alpha between 1e-5 and 1
}

random_search = RandomizedSearchCV(
    estimator=sgd,
    param_distributions=param_dist,
    n_iter=5,           # Try 5 random alpha values
    scoring="roc_auc",
    cv=3,               # 3-fold CV
    n_jobs=-1,
    random_state=12378,
    verbose=2
)

random_search.fit(X_train_svm_scaled, y_train_svm)

# === 7. Best model
best_sgd_model = random_search.best_estimator_
print("\nBest alpha:", random_search.best_params_)
```

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits

Best alpha: {'alpha': np.float64(0.0012742749857031334)}
```

```
# === 8. Predict on test set
y_pred_best = best_sgd_model.predict(X_test_svm_scaled)
y_score_best = best_sgd_model.decision_function(X_test_svm_scaled)

# === 9. Evaluation
print("\n Confusion Matrix:")
print(confusion_matrix(y_test_svm, y_pred_best))

print("\n Classification Report:")
print(classification_report(y_test_svm, y_pred_best, target_names=["no", "yes"]))

auc_best = roc_auc_score(y_test_svm, y_score_best)
print(f"\n Best AUC Score: {auc_best:.4f}")
```

```
 Confusion Matrix:
[[1183393   87489]
 [     39    1600]]

 Classification Report:
              precision    recall  f1-score   support

          no       1.00      0.93      0.96   1270882
         yes       0.02      0.98      0.04      1639

    accuracy                           0.93   1272521
   macro avg       0.51      0.95      0.50   1272521
weighted avg       1.00      0.93      0.96   1272521


 Best AUC Score: 0.9917
```

```
# === 1. Prepare Train/Test
X_train_nn = train_data_smote_balanced.drop(columns=["isFraud"])
y_train_nn = train_data_smote_balanced["isFraud"].map({"no": 0, "yes": 1}).astype(float)

X_test_nn = X_test.copy()
y_test_nn = y_test.replace({'no': 0, 'yes': 1}).astype(float)
```

```
# === 2. Move to PyTorch Tensors
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
# === Fix: Force all features to numeric float32
X_train_nn = X_train_nn.astype('float32')
X_test_nn = X_test_nn.astype('float32')

X_train_tensor = torch.tensor(X_train_nn.values, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train_nn.values.reshape(-1, 1), dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test_nn.values, dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_test_nn.values.reshape(-1, 1), dtype=torch.float32).to(device)

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
```

```
# === 3. Define Neural Network
class FraudNet(nn.Module):
    def __init__(self, input_dim):
        super(FraudNet, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(8, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

model = FraudNet(X_train_nn.shape[1]).to(device)
```

```
# === 4. Compile and Train
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    model.train()
    epoch_loss = 0
    for xb, yb in train_loader:
        preds = model(xb)
        loss = criterion(preds, yb)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {epoch_loss/len(train_loader):.4f}")
```

```
Epoch 1, Loss: 0.0449
Epoch 2, Loss: 0.0337
Epoch 3, Loss: 0.0314
Epoch 4, Loss: 0.0305
Epoch 5, Loss: 0.0295
Epoch 6, Loss: 0.0285
Epoch 7, Loss: 0.0276
Epoch 8, Loss: 0.0271
Epoch 9, Loss: 0.0269
Epoch 10, Loss: 0.0266
```

```
# === 5. Evaluation
model.eval()
with torch.no_grad():
    y_pred_prob_nn = model(X_test_tensor).cpu().numpy()
    y_pred_class_nn = (y_pred_prob_nn >= 0.5).astype(int)

# === 6. Metrics
print("\n Confusion Matrix:")
print(confusion_matrix(y_test_nn, y_pred_class_nn))  # No .cpu()

print("\n Classification Report:")
print(classification_report(y_test_nn, y_pred_class_nn, target_names=["no", "yes"]))  # No .cpu()

f1_nn = f1_score(y_test_nn, y_pred_class_nn)
print(f"\n F1 Score: {f1_nn:.4f}")

auc_nn = roc_auc_score(y_test_nn, y_pred_prob_nn)
print(f"\n AUC Score: {auc_nn:.4f}")
```

```
 Confusion Matrix:
[[1266222    4660]
 [      3    1636]]

 Classification Report:
              precision    recall  f1-score   support

          no       1.00      1.00      1.00   1270882
         yes       0.26      1.00      0.41      1639

    accuracy                           1.00   1272521
   macro avg       0.63      1.00      0.71   1272521
weighted avg       1.00      1.00      1.00   1272521


 F1 Score: 0.4124

 AUC Score: 0.9992
```

```
# === 1. Build Decision Tree on original imbalanced data
dt_model_imbalanced = DecisionTreeClassifier(random_state=123)
dt_model_imbalanced.fit(X_train, y_train)

# === 2. Predict
y_pred_dt = dt_model_imbalanced.predict(X_test)
y_pred_prob_dt = dt_model_imbalanced.predict_proba(X_test)[:,1]

# === 3. Convert y_test to numeric
y_test_dt = y_test.replace({'no': 0, 'yes': 1}).astype(int)

# === 4. Evaluate
print("\n🔵 Decision Tree on Imbalanced Data")
print(confusion_matrix(y_test_dt, y_pred_dt))
print(classification_report(y_test_dt, y_pred_dt))
print("AUC:", roc_auc_score(y_test_dt, y_pred_prob_dt))
```

```
🔵 Decision Tree on Imbalanced Data
[[1270735     147]
 [    227    1412]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00   1270882
           1       0.91      0.86      0.88      1639

    accuracy                           1.00   1272521
   macro avg       0.95      0.93      0.94   1272521
weighted avg       1.00      1.00      1.00   1272521

AUC: 0.9306926237452735
```

```
# === 1. Build Random Forest
rf_model_imbalanced = RandomForestClassifier(
    n_estimators=20,
    random_state=123,
    n_jobs=-1
)
rf_model_imbalanced.fit(X_train, y_train)

# === 2. Predict
y_pred_rf = rf_model_imbalanced.predict(X_test)
y_pred_prob_rf = rf_model_imbalanced.predict_proba(X_test)[:,1]

# === 3. Convert y_test to numeric
y_test_rf = y_test.replace({'no': 0, 'yes': 1}).astype(int)

# === 4. Evaluate
print("\n🔵 Random Forest on Imbalanced Data")
print(confusion_matrix(y_test_rf, y_pred_rf))
print(classification_report(y_test_rf, y_pred_rf))
print("AUC:", roc_auc_score(y_test_rf, y_pred_prob_rf))
```

```
🔵 Random Forest on Imbalanced Data
[[1270824      58]
 [    320    1319]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00   1270882
           1       0.96      0.80      0.87      1639

    accuracy                           1.00   1272521
   macro avg       0.98      0.90      0.94   1272521
weighted avg       1.00      1.00      1.00   1272521

AUC: 0.9883402717135432
```

```
# === 4. (C) Neural Network (Imbalanced)

# Coerce X_train, X_test to float32 for Neural Network later
X_train_fixed = X_train.astype(np.float32)
X_test_fixed = X_test.astype(np.float32)

# Fix y_test labels for evaluation later
y_test_fixed = y_test.replace({'no': 0, 'yes': 1}).astype(int)

# Torch tensors
X_train_tensor = torch.tensor(X_train_fixed.values, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train.replace({'no': 0, 'yes': 1}).values.reshape(-1, 1), dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test_fixed.values, dtype=torch.float32).to(device)

# Dataloader
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)

# Define Model
class FraudNet(nn.Module):
    def __init__(self, input_dim):
        super(FraudNet, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(8, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

nn_model_imbalanced = FraudNet(X_train.shape[1]).to(device)

# Train
criterion = nn.BCELoss()
optimizer = optim.Adam(nn_model_imbalanced.parameters(), lr=0.001)

for epoch in range(10):
    nn_model_imbalanced.train()
    for xb, yb in train_loader:
        preds = nn_model_imbalanced(xb)
        loss = criterion(preds, yb)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# === Evaluate Imbalanced Neural Network
nn_model_imbalanced.eval()
with torch.no_grad():
    y_pred_prob_nn_imbal = nn_model_imbalanced(X_test_tensor).cpu().numpy()
    y_pred_class_nn_imbal = (y_pred_prob_nn_imbal >= 0.5).astype(int)

print("\n Neural Network on Imbalanced Data")
print(confusion_matrix(y_test_fixed, y_pred_class_nn_imbal))
print(classification_report(y_test_fixed, y_pred_class_nn_imbal, target_names=["no", "yes"]))
print("AUC:", roc_auc_score(y_test_fixed, y_pred_prob_nn_imbal))
```

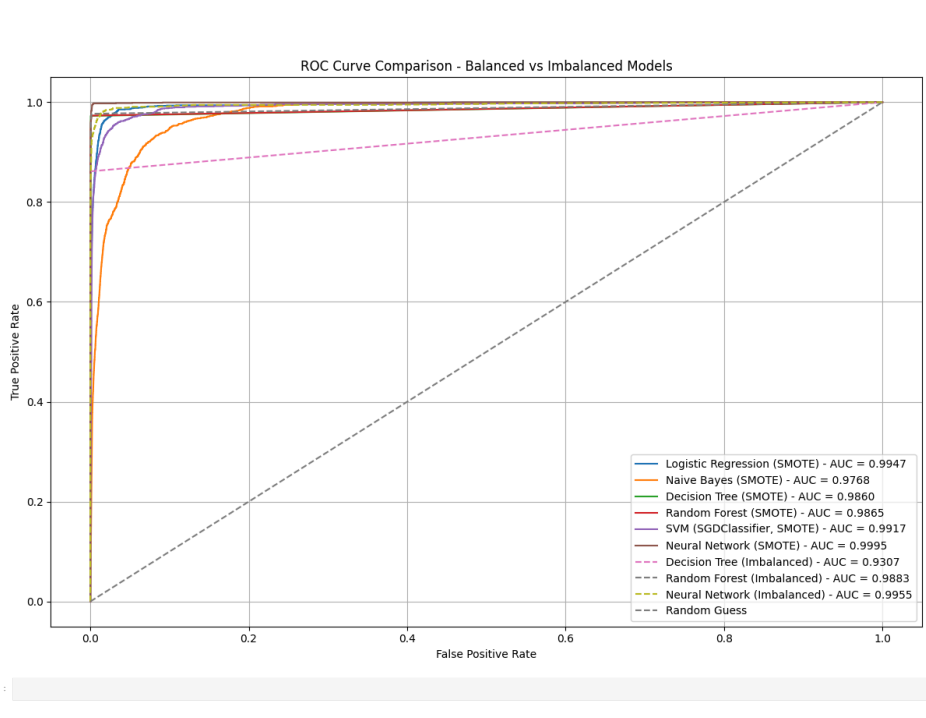
```
 Neural Network on Imbalanced Data
[[1270819      63]
 [    429    1210]]
              precision    recall  f1-score   support

          no       1.00      1.00      1.00   1270882
         yes       0.95      0.74      0.83      1639

    accuracy                           1.00   1272521
   macro avg       0.98      0.87      0.92   1272521
weighted avg       1.00      1.00      1.00   1272521

AUC: 0.9955431160552655
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

# === 1. Fresh list to collect results
results = []

# === 2. Define evaluation function
def evaluate_model(name, y_true, y_pred, y_prob):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    auc = roc_auc_score(y_true, y_prob)

    results.append({
        "Model": name,
        "Accuracy": round(accuracy, 4),
        "Precision": round(precision, 4),
        "Recall": round(recall, 4),
        "F1 Score": round(f1, 4),
        "AUC": round(auc, 4)
    })

# === 3. Make sure all y_test are numeric (0/1)

# Balanced Models
y_test_nb_fixed = y_test.replace({'no': 0, 'yes': 1}).astype(int)
y_test_tree_fixed = y_test.replace({'no': 0, 'yes': 1}).astype(int)
y_test_rf_fixed = y_test_rf_final    # already numeric
y_test_svm_fixed = y_test_svm        # already numeric
y_test_nn_fixed = y_test_nn          # already numeric

# Imbalanced Models
y_test_imbalanced = y_test.replace({'no': 0, 'yes': 1}).astype(int)

# === 4. Evaluate all models

# === Balanced Models (SMOTE applied)
evaluate_model("Logistic Regression (SMOTE)", y_test_encoded, y_pred_class, y_pred_prob)
evaluate_model("Naive Bayes (SMOTE)", y_test_nb_fixed, nb_pred_class, nb_pred_prob)
evaluate_model("Decision Tree (SMOTE)", y_test_tree_fixed, tree_pred, tree_prob)
evaluate_model("Random Forest (SMOTE)", y_test_rf_fixed, rf_pred_class, rf_pred_prob)
evaluate_model("SVM (SMOTE, SGDClassifier)", y_test_svm_fixed, y_pred_best, y_score_best)
evaluate_model("Neural Network (SMOTE)", y_test_nn_fixed, y_pred_class_nn_bal, y_pred_prob_nn_bal)

# === Imbalanced Models (no SMOTE)
evaluate_model("Decision Tree (Imbalanced)", y_test_imbalanced, y_pred_dt, y_pred_prob_dt)
evaluate_model("Random Forest (Imbalanced)", y_test_imbalanced, y_pred_rf, y_pred_prob_rf)
evaluate_model("Neural Network (Imbalanced)", y_test_imbalanced, y_pred_class_nn_imbal, y_pred_prob_nn_imbal)

# === 5. Create final summary table
summary_table = pd.DataFrame(results)

print("\n Model Performance Summary (Balanced + Imbalanced):")
display(summary_table)
```

```
 Model Performance Summary (Balanced + Imbalanced):
```

	Model	Accuracy	Precision	Recall	F1 Score	AUC
0	Logistic Regression (SMOTE)	0.9679	0.0379	0.9799	0.0729	0.9947
1	Naive Bayes (SMOTE)	0.9601	0.0262	0.8279	0.0507	0.9768
2	Decision Tree (SMOTE)	0.9995	0.7132	0.9725	0.8229	0.9860
3	Random Forest (SMOTE)	0.9997	0.9609	0.8090	0.8784	0.9865
4	SVM (SMOTE, SGDClassifier)	0.9312	0.0180	0.9762	0.0353	0.9917
5	Neural Network (SMOTE)	0.9973	0.3193	0.9957	0.4836	0.9995
6	Decision Tree (Imbalanced)	0.9997	0.9057	0.8615	0.8831	0.9307
7	Random Forest (Imbalanced)	0.9997	0.9579	0.8048	0.8747	0.9883
8	Neural Network (Imbalanced)	0.9996	0.9505	0.7383	0.8310	0.9955

```
from sklearn.metrics import roc_curve

# === 1. Compute FPR, TPR for each model
fpr_log, tpr_log, _ = roc_curve(y_test_encoded, y_pred_prob)                    # Logistic Regression
fpr_nb, tpr_nb, _ = roc_curve(y_test_nb_fixed, nb_pred_prob)                    # Naive Bayes
fpr_tree, tpr_tree, _ = roc_curve(y_test_tree_fixed, tree_prob)                 # Decision Tree (SMOTE)
fpr_rf, tpr_rf, _ = roc_curve(y_test_rf_fixed, rf_pred_prob)                    # Random Forest (SMOTE)
fpr_svm, tpr_svm, _ = roc_curve(y_test_svm_fixed, y_score_best)                 # SVM
fpr_nn, tpr_nn, _ = roc_curve(y_test_nn_fixed, y_pred_prob_nn_bal)              # Neural Network (SMOTE)

# Imbalanced Models
fpr_dt_imb, tpr_dt_imb, _ = roc_curve(y_test_imbalanced, y_pred_prob_dt)          # Decision Tree (Imbalanced)
fpr_rf_imb, tpr_rf_imb, _ = roc_curve(y_test_imbalanced, y_pred_prob_rf)          # Random Forest (Imbalanced)
fpr_nn_imb, tpr_nn_imb, _ = roc_curve(y_test_imbalanced, y_pred_prob_nn_imbal)    # Neural Network (Imbalanced)

# === 2. Plot all ROC curves
plt.figure(figsize=(12, 8))

# SMOTE Models
plt.plot(fpr_log, tpr_log, label=f"Logistic Regression (SMOTE) - AUC = {summary_table.loc[summary_table['Model']=='Logistic Regression (SMOTE)', 'AUC'].values[0]:.4f}")
plt.plot(fpr_nb, tpr_nb, label=f"Naive Bayes (SMOTE) - AUC = {summary_table.loc[summary_table['Model']=='Naive Bayes (SMOTE)', 'AUC'].values[0]:.4f}")
plt.plot(fpr_tree, tpr_tree, label=f"Decision Tree (SMOTE) - AUC = {summary_table.loc[summary_table['Model']=='Decision Tree (SMOTE)', 'AUC'].values[0]:.4f}")
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest (SMOTE) - AUC = {summary_table.loc[summary_table['Model']=='Random Forest (SMOTE)', 'AUC'].values[0]:.4f}")
plt.plot(fpr_svm, tpr_svm, label=f"SVM (SGDClassifier, SMOTE) - AUC = {summary_table.loc[summary_table['Model']=='SVM (SMOTE, SGDClassifier)', 'AUC'].values[0]:.4f}")
plt.plot(fpr_nn, tpr_nn, label=f"Neural Network (SMOTE) - AUC = {summary_table.loc[summary_table['Model']=='Neural Network (SMOTE)', 'AUC'].values[0]:.4f}")

# Imbalanced Models
plt.plot(fpr_dt_imb, tpr_dt_imb, linestyle='--', label=f"Decision Tree (Imbalanced) - AUC = {summary_table.loc[summary_table['Model']=='Decision Tree (Imbalanced)', 'AUC'].values[0]
plt.plot(fpr_rf_imb, tpr_rf_imb, linestyle='--', label=f"Random Forest (Imbalanced) - AUC = {summary_table.loc[summary_table['Model']=='Random Forest (Imbalanced)', 'AUC'].values[0]
plt.plot(fpr_nn_imb, tpr_nn_imb, linestyle='--', label=f"Neural Network (Imbalanced) - AUC = {summary_table.loc[summary_table['Model']=='Neural Network (Imbalanced)', 'AUC'].values[0

# === 3. Random line
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label="Random Guess")

# === 4. Final touches
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison - Balanced vs Imbalanced Models")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.show()
```


ROC Curve Comparison - Balanced vs Imbalanced Models
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.2
0.4
0.6
0.8
1.0
Logistic Regression (SMOTE) - AUC = 0.9947
Naive Bayes (SMOTE) - AUC = 0.9768
Decision Tree (SMOTE) - AUC = 0.9860
Random Forest (SMOTE) - AUC = 0.9865
SVM (SGDClassifier, SMOTE) - AUC = 0.9917
Neural Network (SMOTE) - AUC = 0.9995
Decision Tree (Imbalanced) - AUC = 0.9307
Random Forest (Imbalanced) - AUC = 0.9883
Neural Network (Imbalanced) - AUC = 0.9955
Random Guess

In [ ]: